

PRACTICAL Week 4A — Multiple Child Processes

This program creates multiple child processes using `fork()` and displays the PID of each child.

It demonstrates how a parent process can create and manage multiple child processes. Your uploaded Practical 3 and 4 file also contains a multiple-child program.

```
sri_dhruthi@Dhruthi: ~  
GNU nano 8.7.1 week4a.c *  
#include <stdio.h>  
#include <unistd.h>  
#include <sys/wait.h>  
  
int main()  
{  
    int i;  
    pid_t pid;  
  
    for (i = 1; i <= 3; i++)  
    {  
        pid = fork();  
  
        if (pid == 0)  
        {  
            printf("Child %d PID = %d\n", i, getpid());  
            return 0;  
        }  
    }  
  
    for (i = 1; i <= 3; i++)  
    {  
        wait(NULL);  
    }  
  
    printf("Parent process completed.\n");  
  
    return 0;  
}
```

```
sri_dhruthi@Dhruthi:~$ nano week4a.c  
sri_dhruthi@Dhruthi:~$ gcc week4a.c -o week4a  
sri_dhruthi@Dhruthi:~$ ./week4a  
Child 1 PID = 878  
Child 3 PID = 880  
Child 2 PID = 879  
Parent process completed.
```

Week 4B — wait() and waitpid()

This program demonstrates synchronization of multiple child processes using both wait() and waitpid().

```
sri_dhruthi@Dhruthi: ~  
GNU nano 8.7.1 week4b.c *  
#include <stdio.h>  
#include <unistd.h>  
#include <sys/wait.h>  
  
int main()  
{  
    pid_t child1, child2;  
    int status;  
  
    child1 = fork();  
  
    if (child1 == 0)  
    {  
        printf("Child 1 running, PID = %d\n", getpid());  
        sleep(2);  
        printf("Child 1 finished\n");  
        return 10;  
    }  
  
    child2 = fork();  
  
    if (child2 == 0)  
    {  
        printf("Child 2 running, PID = %d\n", getpid());  
        sleep(4);  
        printf("Child 2 finished\n");  
        return 20;  
    }  
  
    wait(NULL);  
    printf("Parent used wait()\n");  
  
    waitpid(child2, &status, 0);  
    printf("Parent used waitpid()\n");  
  
    printf("Parent process completed.\n");  
}
```

```
Parent process completed.  
sri_dhruthi@Dhruthi:~$ nano week4b.c  
sri_dhruthi@Dhruthi:~$ gcc week4b.c -o week4b  
sri_dhruthi@Dhruthi:~$ ./week4b  
Child 2 running, PID = 963  
Child 1 running, PID = 962  
Child 1 finished  
Parent used wait()  
Child 2 finished  
Parent used waitpid()  
Parent process completed.
```

Week 4C — Zombie Process

This program demonstrates a zombie process by allowing the child to terminate while the parent temporarily does not call `wait()`.

```
sri_dhruthi@Dhruthi: ~  
GNU nano 8.7.1 week4c.c *  
#include <stdio.h>  
#include <unistd.h>  
#include <sys/wait.h>  
  
int main()  
{  
    pid_t pid = fork();  
  
    if (pid == 0)  
    {  
        printf("Child process exiting...\n");  
        return 0;  
    }  
    else  
    {  
        printf("Parent PID = %d\n", getpid());  
        printf("Child PID = %d\n", pid);  
  
        sleep(10);  
  
        wait(NULL);  
  
        printf("Zombie removed. Parent completed.\n");  
    }  
  
    return 0;  
}
```

```
Parent process completed.  
sri_dhruthi@Dhruthi:~$ nano week4c.c  
sri_dhruthi@Dhruthi:~$ gcc week4c.c -o week4c  
sri_dhruthi@Dhruthi:~$ ./week4c  
Parent PID = 1009  
Child PID = 1010  
Child process exiting...  
Zombie removed. Parent completed.  
sri_dhruthi@Dhruthi:~$ |
```